Lab Sheet 03 · Part A Answers

HDIT 12103 Website Design and Development · Interactive Forms and Client-side Validation

Task A1: What a form does by itself

1. After the **?**, the address bar shows:
`fullName=Muzawir+Rizwan&studentId=26015&email=rizwanmuzawir%40gmail.com&phone=0720470471&workshop=web&session=morning&guests=02&laptop=on`
Spaces become **+**, and the @ in the email becomes **%40** (URL encoding).

2. In my run **no field is missing**, because I ticked the laptop checkbox, so `laptop=on` was sent. If the **laptop** checkbox is left unticked, it is the one field that disappears from the address bar: browsers only send a checkbox when it is ticked. An unticked checkbox is left out completely, unlike an empty text box, which is still sent as `name=`.

3. No written answer. Add the listener and check that the page stays put and the Console shows `Form submitted`.

4. Yes. Pressing Enter in a text box submits the form, because the form has a `type="submit"` button.

Task A2: Predict, then run

#	Code	Your prediction	Actual result
1	`getElementById("full-name").value`	"Muzawir Rizwan" (spaces kept, if any)	"Muzawir Rizwan"
2	`getElementById("full-name").value.trim()`	"Muzawir Rizwan" (spaces removed)	"Muzawir Rizwan"
3	`getElementById("guests").value`	"02" (a string)	"02" (a string)
4	`getElementById("guests").value + 1`	"021" (joins as text)	"021" (string joining)
5	`Number(getElementById("guests").value) + 1`	3	3
6	`getElementById("laptop").checked`	true	true
7	`querySelector('input[name="session"]:checked').value`	"morning"	"morning"
8	`getElementById("workshop").value`	"web" (the value, not the words)	"web"
9	`new FormData(getElementById("reg-form")).get("laptop")`	"on" (box is ticked)	"on" (ticked checkbox is sent as "on")

Notes: Row 1 assumes the name was typed with no extra spaces; if you typed spaces around it, row 1 keeps them and row 2 removes them. Row 4 gives `"021"` because the value `"02"` is text, so `+ 1` joins instead of adds. The sheet writes `"regform"` in row 9, but the real id is `"reg-form"`.

Row 8 gives `web`**, not Responsive web design. Where does each one come from?**
From the option: `<option value="web">Responsive web design</option>`. `web` is the `value` attribute, which `select.value` returns. "Responsive web design" is the text between the tags, which the user sees and which `select.options[select.selectedIndex].text` returns.

Task A3: Check table

Check	Expected / result
#status	Thank you, Muzawir Rizwan!

Check	Expected / result
Summary list	Name: Muzawir Rizwan · Workshop: Responsive web design · Session: morning · Guests: 2 · Own laptop: Yes
The form	Empty again, Session back to Morning
Address bar	Unchanged: no ? and no reload

Step 4 comparison: the Console log (before the reset) shows `workshop: "web"`, `guests: "02"` (a string) and `laptop: "on"`. The summary shows "Responsive web design", the number 2 and "Yes". The summary converts and formats the values; `FormData` holds the raw ones.

Lab Sheet 03 · Part B Answers

HDIT 12103 Website Design and Development · Validation

Task B1: Let the browser check

Step 2. What the browser does (message wording differs between browsers; these are Chrome-style):

Try	What the browser does
Full name: Ab	Blocks the submit and shows a bubble: please lengthen this text to 3 characters or more (you are using 2).
Student ID left empty	Blocks the submit: "Please fill out this field." (required)
Email: rizwanmuzawir@gmail	Accepts it. No bubble for the email field, because type="email" allows an address with no dot after the @.
Phone: 12345	Blocks the submit: "Please match the requested format." (pattern)
Guests: 5	Blocks the submit: "Value must be less than or equal to 3." (max)

Step 3. The browser accepts rizwanmuzawir@gmail because the HTML email type only needs something@something. On a campus registration list this lets through addresses that are not real or deliverable, so confirmation emails would bounce and the organisers could not contact the student. This is one reason to add our own regex check in JavaScript.

Task B2.1: Predict, then run

#	Code	Your prediction	Actual
1	`/^07\d{8}$/.test("0771234567")`	true	true
2	`/^07\d{8}$/.test("077 123 4567")`	false (has spaces)	false
3	`/^07\d{8}$/.test("771234567")`	false (no 07, only 9 digits)	false
4	`/^07\d{8}$/.test("07712345678")`	false (11 digits, too long)	false
5	`/^HD\d-\d{3}$/.test("HD3-017")`	true	true
6	`/^HD\d-\d{3}$/.test("hd3-017")`	false (case-sensitive)	false
7	`/^HD\d-\d{3}$/.test("HD3-17")`	false (only 2 digits)	false
8	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("muzawir@icst.lk")`	true	true
9	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("muzawir@icst")`	false (no dot after @)	false
10	`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("muzawir rizwan@icst.lk")`	false (contains a space)	false

Remove ^ and $ from row 4's pattern: `/07\d{8}/.test("07712345678")` changes from **false** to **true**. Without ^ and $ the regex only looks for 07 followed by 8 digits *anywhere* in the text, so it finds a match in the first 10 characters and ignores the extra digit. ^ and $ force the whole value to match, not just part of it.

Same checks with my own details:

Code	Result	Why
`/^07\d{8}$/.test("0720470471")`	true	My mobile: 07 then 8 digits
`/^HD\d-\d{3}$/.test("26015")`	false	No HD prefix and no dash, so my Part A student ID would fail here

Code	Result	Why
`/^HD\d-\d{3}$/.test("HD3-017")`	true	The sheet's example format passes
`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("rizwanmuzawir@gmail.com")`	true	My email has a dot after the @
`/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test("rizwanmuzawir@gmail")`	false	No dot after the @

Task B2.2: Validate every field

The code is in `js/part-b.js`. Test it on `part-b.html`, which loads `part-b.js` and has `novalidate` on the form tag. Expected test results:

Test	Expected
Submit the empty form	Five red messages (name, student ID, email, phone, workshop); guests is fine at 0. The cursor jumps to Full name.
Valid except phone "072 047 0471"	Only the phone message shows. The cursor jumps to the phone box.
Guests 2.5, then -1, then 3	Error, error, no error.
Everything valid (Muzawir Rizwan, HD3-017, rizwanmuzawir@gmail.com, 0720470471, Responsive web design, 2 guests)	Registration complete! and the form empties.

Task B3: Live feedback

Do this	Result
Type riz in the email box	No error yet: the field has not been left, so the blur event has not fired.
Press Tab to leave the box	Enter an email like name@example.com.
Click back and finish typing wanmuzawir@gmail.com	The error disappears while you type, because the input event now rechecks the field.

B3.2 (Challenge): showError also toggles a `valid` class, so a touched field with no error gets a green border. Add this CSS: `.valid { border: 2px solid #2e7d32; }`

Task B4: Check it without a mouse

1. Each input now has `aria-describedby="…-error"`, pointing to its error span (visible in the Elements panel).

2. Anything you could not do: *nothing; every field can be reached with Tab and Shift+Tab, the radio buttons and select work with the arrow keys, Space ticks the checkbox and Enter submits.* (Confirm this in your own run and write down anything different.)

3. With a wrong phone number the cursor goes to the **phone box**, because the submit handler calls `firstBad.focus()`. This helps a keyboard user because they do not have to Tab through the whole form to find the problem, and a screen reader then reads the field together with its error message.

Task B5: Break your own validation

3. Yes, the invalid email was sent: the address bar shows `?fullName=&…&email=not-an-email&…`. Our validation did not run because `form.submit()` sends the form directly and does **not** fire the `submit` event, so our listener never ran (it also skips the browser's own checks).

4. Two other ways to get past the checks:
• Turn off JavaScript in the browser, or edit the page in DevTools (remove `novalidate`, `pattern` or change the values) before submitting.
• Skip the page entirely and send the request with a tool such as curl or Postman. This is why the server must validate every value again.

Lab Sheet 03 · Part C Answers

HDIT 12103 Website Design and Development · Mini project: Sign-up form

Files

`signup.html` goes in the project root. `signup.js` goes in the `js` folder.

Step 2: The five TODOs

TODO 1: checkUsername

```
function checkUsername(value) {
  if (!/^\w{3,15}$/.test(value)) {
    return "Use 3 to 15 letters, numbers or _.";
  }
  if (takenUsernames.includes(value.toLowerCase())) {
    return "That username is taken. Try another.";
  }
  return "";
}
```

TODO 2: checkEmail

```
function checkEmail(value) {
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value)) {
    return "Enter an email like name@example.com.";
  }
  return "";
}
```

TODO 3: checkPasswordRules

```
function checkPasswordRules(value) {
  const results = {
    length: value.length >= 8,
    number: /\d/.test(value),
    upper: /[A-Z]/.test(value),
    symbol: /[^A-Za-z0-9]/.test(value)
  };
  let metCount = 0;
  for (const key in results) {
    ruleItems[key].classList.toggle("met", results[key]);
    if (results[key]) metCount++;
  }
  showStrength(value, metCount);   // Challenge 1
  return metCount === 4;
}
```

TODO 4: checkConfirm

```
function checkConfirm() {
  if (confirmBox.value === "") return "Please type your password again.";
  if (confirmBox.value !== password.value) return "Passwords do not match.";
  return "";
}
```

TODO 5: Show / Hide button

```
showBtn.addEventListener("click", () => {
  const hidden = password.type === "password";
  password.type = hidden ? "text" : "password";
  showBtn.textContent = hidden ? "Hide" : "Show";
});
```

Step 3: Test table

Test	Expected result	Result
Open the page	Create account is disabled; all four rules show X in grey	Pass

Test	Expected result	Result
Username: Muzawir	That username is taken. Try another.	Pass
Username: ab	Use 3 to 15 letters, numbers or _.	Pass
Username: muzawir_r	No error	Pass
Email: rizwanmuzawir@gmail	Enter an email like name@example.com.	Pass
Password: icst2026	Two rules ticked: 8 characters and a number	Pass
Password: Icst2026!	All four rules ticked	Pass
Confirm: Icst2026	Passwords do not match.	Pass
Click Show, then Hide	The password appears as text, then as dots again	Pass
Confirm Icst2026! and tick the box	Create account becomes enabled	Pass
Click Create account	Welcome, muzawir_r! Your account is ready.; the form disappears; the password is not shown anywhere	Pass

"Muzawir" is in the pretend taken-username list (the `takenUsernames` array in the code), so it shows the taken message, and capitals are ignored. My own username for the passing tests is **muzawir_r** and my email is **rizwanmuzawir@gmail.com**. The results above were checked by running the code against these steps. Repeat them in your own browser and tick each row.

Full sign-up run with my own details

Field	What I typed	Result
Username	muzawir_r	No error (9 characters, letters and _ only, not in the taken list)
Email	rizwanmuzawir@gmail	Enter an email like name@example.com. (no dot after the @)
Email (corrected)	rizwanmuzawir@gmail.com	No error
Date of birth	left empty (optional)	No error; a date under 16 years ago would show the age error
Password	Icst2026! (the sheet's test password)	All four rules ticked; Strength: Strong
Confirm password	Icst2026!	No error (matches)
Campus IT rules box	ticked	Create account becomes enabled
Click Create account		Welcome, muzawir_r! Your account is ready.; the form disappears

Refresh the page after this run: the username muzawir_r is filled in again, because Challenge 4 remembers it in localStorage (the password is never stored). The Start again button clears it.

Task C: Challenge tasks (all five done)

#	Challenge	How it works
1	Strength word	`showStrength()` counts the rules met: Weak (0 to 2), Fair (3), Strong (4), shown under the checklist.
2	Caps Lock warning	A `keydown` listener on the password boxes uses `e.getModifierState("CapsLock")` and shows "Caps Lock is on."; it clears on blur.

#	Challenge	How it works
3	Date of birth 16+	`checkDob()` works out the age from the date and gives an error for anyone under 16 or a future date. The field is optional, so the test table above still passes.
4	Remember username	Only the username is saved in `localStorage` (key `signup-username`) and filled in on reload. The password is never stored.
5	Start again button	Resets the form, clears every error message, unticks all rules, clears the strength word and the remembered username, and shows the form again.

Security note: passwords

The code never shows the password back to the user, never calls `console.log` on it, and never saves it in `localStorage`. Only the username is stored. The `takenUsernames` list is pretend: a real site must ask the server, because only the server knows every account.

Commit message

Complete Part C mini project